

Process Scheduling Simulator

Implementation Plan

OS Lab 3 — IIT Dharwad

Objective

Build a C++ simulator that implements 3 scheduling algorithms (**FIFO**, **Round Robin**, and **MLFQ**) for both single-CPU and dual-CPU configurations.

Usage

```
./scheduler <algorithm> <workload-file> [num_cpus]
```

Algorithms

- **FIFO** — First In First Out (non-preemptive)
- **RR:<quantum>** — Round Robin with given time quantum (e.g., **RR:10**)
- **MLFQ** — Multi-Level Feedback Queue, no priority boost
- **MLFQ:<boost>** — MLFQ with priority boost every <boost> time units (e.g., **MLFQ:20**)

Optional Arguments

- **num_cpus:**
 - 1 (*default*) — Single CPU
 - 2 — Dual CPU (Part II)

Output Format

1. **Schedule output format** (as specified in `schedule_format.txt`):

```
CPU0  
P<pid>,<burst_num> <start_time> <end_time>
```

2. **Average Turnaround Time**
3. **Maximum Turnaround Time**
4. **Simulator Run Time** (wall-clock, excluding file I/O)

Language & Tools Used

- **Language:** C++ (C++17 standard)
- **Compiler:** g++ with `-O2 -std=c++17`
- **Libraries:** Standard Library only (no external dependencies)
 - `<iostream>`, `<fstream>`, `<sstream>` — I/O and file parsing
 - `<vector>`, `<deque>`, `<string>` — containers
 - `<algorithm>` — sorting (for deterministic I/O completion order)
 - `<climits>` — `INT_MAX` sentinel for FIFO quantum
 - `<chrono>` — high-resolution clock for execution timing
 - `<iomanip>` — formatted output (`setprecision`)
- **Source File:** `code.cpp` (all logic in a single file, ~490 lines)
- **Build Command:**

```
g++ -O2 -std=c++17 code.cpp -o scheduler
```

Approach & Design Decisions

1. Data Structures

Designed structs cleanly separating input data from runtime state:

- **Burst** { type (CPU/IO), duration }
 - Represents one burst. Bursts alternate CPU → I/O → CPU → ... parsed from the workload file.
- **Process** { pid, arrival_time, bursts[], runtime_state }
 - Holds the static burst list and mutable runtime state:
 - * `current_burst_idx`: Current burst index.
 - * `remaining`: Remaining time in the current burst.
 - * `completion_time`: Timestamp when the process finished.
 - * `queue_level`: MLFQ queue index (0 = highest, 2 = lowest).
 - * `cpu_burst_number`: Counter tracking CPU burst sequence number for schedule logging.
- **ScheduleEntry** { pid, cpu_burst_number, cpu_id, start_time, end_time }
 - Recorded whenever a process stops running (burst completed or preempted).

- **WaitingProcess** { `proc*`, `io_finish_time` }
 - Tracks processes performing I/O with known completion times.
- **CPUState** { `running*`, `quantum_remaining`, `run_start_time` }
 - Tracks the state of each CPU core (supports 1 or 2 CPUs).

Ready Queues

- **FIFO & RR**: Single `std::deque<Process*>` ready queue.
- **MLFQ**: 3 separate queues `std::deque<Process*>` (Q0, Q1, Q2).

2. Simulation Engine — Tick-by-Tick Approach

A tick-based simulation model (advancing time by 1 unit each step) is used for deterministic execution and simplified state handling.

Each tick cycle executes in the following sequence:

1. Step 1: Arrivals

- Check if new processes arrive at the current timestamp.
- Enqueue them into the ready queue with the first CPU burst initialized.

2. Step 2: I/O Completions

- Check waiting list for processes whose I/O finishes at the current tick.
- Sort by (`finish_time`, `pid`) for deterministic ordering.
- Re-enqueue into the ready queue for their next CPU burst.

3. Step 3: MLFQ Priority Boost (if enabled)

- Every boost interval (e.g., every 20 ticks), promote all processes in Q1 and Q2 back to Q0.
- Reset running processes' priority levels to prevent starvation.

4. Step 4: CPU Assignment

- For each idle CPU, pick the next available process from the ready queue.
- For MLFQ, pick from the highest non-empty priority queue (Q0 → Q1 → Q2).

5. Step 5: Fast-Forward Optimization

- If all CPUs are idle:
 - If no I/O is pending → Jump time directly to the next process arrival.
 - If I/O is pending → Jump time to the earliest I/O completion.

6. Step 6: Advance Time (time++)

- Decrement remaining burst time and quantum for running processes.
- Handle completion & preemption:
 - **Burst completes** (remaining == 0):
 - * Log schedule entry.
 - * If more bursts remain → move to I/O waiting queue.
 - * If no more bursts remain → mark process finished and record completion time.
 - **Quantum expires** (quantum_remaining == 0):
 - * Log schedule entry.
 - * Preempt process and push back to ready queue.
 - * For MLFQ: Demote to the next lower queue (Q0 → Q1 → Q2).

3. Algorithm-Specific Details

A. FIFO (First In First Out)

- **Non-preemptive:** Quantum set to INT_MAX.
- Process executes its entire CPU burst without interruption.
- Ready queue behaves as a standard FCFS queue.
- Upon burst completion, the process transitions to I/O or terminates.

B. Round Robin (RR)

- **Preemptive:** Configurable time quantum.
- When quantum expires, the process is moved to the back of the ready queue.
- Progress in the current CPU burst is preserved (remaining time is not reset).
- Evaluated with quantum values: 2, 5, 10, 50.

C. MLFQ (Multi-Level Feedback Queue)

- **3 Priority Queues:** Q0 (highest) → Q1 → Q2 (lowest).
- All queues utilize a time quantum of 2.
- New processes enter at Q0.
- Scheduler always selects from the highest non-empty queue.
- **Demotion on Quantum Expiry:** Process drops to next lower priority (Q0 → Q1, Q1 → Q2, Q2 remains at Q2).

- **I/O Retention:** Processes voluntarily yielding for I/O maintain their current queue level (rewarding I/O-bound jobs).
- **Periodic Priority Boost:** Every N ticks, all processes are promoted to Q0 (tested with boost intervals like 20 ticks).

4. Dual-CPU Extension (Part II)

- Maintained as `std::vector<CPUState>` of size `num_cpus` (1 or 2).
- Both CPUs share the same ready queue / MLFQ structure.
- At each tick, idle CPUs independently pick the next process.
- Schedule entries include `cpu_id` for distinct per-CPU schedule logging.
- I/O remains independent and shared across all CPUs.

5. Output Format & Statistics Example

```
CPU0
P1,1      0      200      <- pid=1, cpu_burst=1, ran from t=0 to t=200
P2,1      200     300
CPU1
P3,1      0      100      <- only shown when num_cpus = 2

Average Turnaround Time: 363.33
Maximum Turnaround Time: 590
Simulator Run Time: 0.005 ms
```

$$\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$$

6. Timing Measurement

Utilizes C++ `<chrono>` `high_resolution_clock` to measure strictly the simulation execution (`simulate()` function call), excluding file I/O overhead to capture the pure algorithmic runtime.

Test Cases & Verified Results

Test workload files from `assignment3_test_cases/`:

- `process1.txt` — 3 CPU-only processes (no I/O)
- `process2.txt` — 4 processes with identical CPU+I/O burst patterns
- `process3.txt` — 4 processes with very short CPU bursts and longer I/O
- `process4.txt` — 3 processes with mixed burst lengths and staggered arrivals
- `process5.txt` — 6 processes (extended version of `process4`)

Algorithm	Test File	Avg TAT	Max TAT
FIFO (1 CPU)	process1.txt	363.33	590
FIFO (1 CPU)	process2.txt	525.00	600
RR:10 (1 CPU)	process1.txt	453.33	590
RR:2 (1 CPU)	process2.txt	595.00	600
MLFQ (1 CPU)	process4.txt	132.33	144
MLFQ:20 (1 CPU)	process4.txt	133.00	147
FIFO (2 CPU)	process1.txt	230.00	390
RR:10 (2 CPU)	process5.txt	106.67	120
MLFQ:20 (2 CPU)	process5.txt	108.00	121

File Structure

```

lab3/
|-- code.cpp           <- Main simulator source (C++)
|-- scheduler.exe      <- Compiled binary
|-- plan.txt           <- Plain text implementation plan
|-- plan.md            <- Markdown implementation plan
|-- schedule_format.txt <- Expected output format reference
|-- 3_process_scheduling.pdf <- Assignment PDF
|-- 3_process_scheduling.txt <- Assignment text (extracted)
'-- assignment3_test_cases-.../
    '-- assignment3_test_cases/
        |-- process1.txt
        |-- process2.txt
        |-- process3.txt
        |-- process4.txt
        '-- process5.txt

```